



**PROYECTO INTEGRADOR CRONOGRAMA DE ACUERO ALAS 7 SEMANAS APLICANDO EN
JUEGO PIEDRA, PAPEL O TIJERA**

Estudiante

RAMIREZ CALVA ROBINSON ALEXANDER

Materia

LOGICA DE PROGRAMACION

Proyecto Python: Juego de Piedra, Papel o Tijera

Contenido del Curso y Cronograma Académico

01 Cronograma y Objetivos del Proyecto

02 Semana 1: Fundamentos de Resolución de Problemas

03 Semana 2: Entorno de Desarrollo y Depuración

04 Semana 3: Almacenamiento de Datos y Variables

05 Semana 4: Algoritmos y Diagramas de Flujo

06 Semana 5: Estructuras de Decisión Controladas

07 Semana 6: Bucles e Interactividad

08 Semana 7: Estructuras de Datos y Optimización

Semana 1: Resolución de Problemas

Definición de problemas computacionales y diseño de soluciones lógicas para el proyecto inicial.

Fundamentos



Un problema computacional requiere definir entradas y salidas claras. Se distinguen problemas algorítmicos y heurísticos.

Conceptos Clave

Pasos Solución



El proceso incluye el análisis profundo del problema, el diseño detallado del algoritmo y la codificación con pruebas.

Metodología

Lógica Juego



Definimos la interacción: el usuario elige opción, la PC otra y se comparan bajo las reglas establecidas del juego.

Estructura

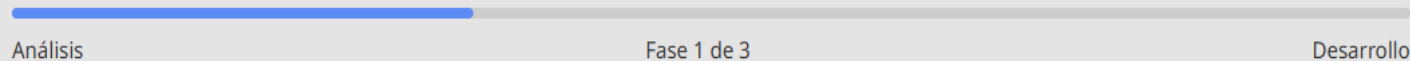
Reglas Básicas



Piedra vence a tijera, tijera vence a papel y papel vence a piedra. Estas reglas definen la lógica de victoria.

Aplicación

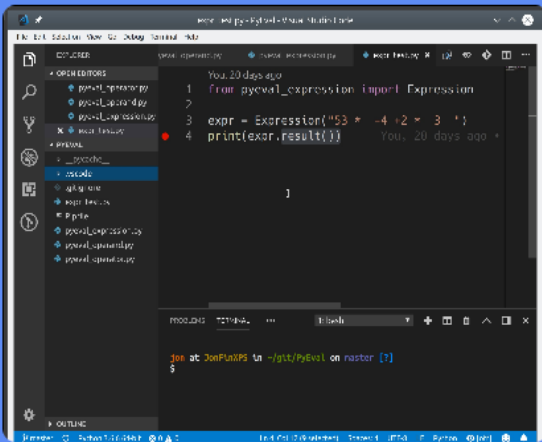
Progreso del Proyecto: Fase Inicial



Iniciar Algoritmo

Semana 2: Entorno de Desarrollo y Depuración

Configuración de herramientas esenciales e introducción a la programación práctica en Python.



Configuración Inicial

Instalación y uso de VS Code, IDLE o Replit para comenzar a escribir código Python de manera profesional.

Proceso de Depuración



Aprendemos a identificar y corregir errores de sintaxis para asegurar que el programa funcione correctamente.

Primer Código



Uso de la función `print` para mostrar mensajes en la consola y verificar la ejecución lógica del sistema.



Resultado Esperado

Ejecución exitosa del primer script confirmando que el entorno está listo para el desarrollo del juego.

Semana 3: Almacenamiento de Datos y Variables

Proyecto Python: Juego de Piedra, Papel o Tijera. Fundamentos de lógica y manejo de información.

Tipos de Datos



Uso de variables y tipos de datos esenciales (str, int) para capturar y guardar las elecciones de los jugadores eficazmente.

[Ver Tipos](#)

Lógica de Juego



Implementación de operadores y prioridad de operaciones para determinar el ganador mediante la lógica de programación.

[Ver Tutorial](#)



Módulo Random

Permite que la computadora realice una elección azarosa entre las opciones disponibles del juego de forma automática.



Piedra



Papel



Tijera

[Importar Random](#)

Elección IA



Introducción práctica al módulo random para simular la toma de decisiones de la computadora en cada partida disputada.

[Ver Ejemplo](#)

Prioridad Operativa



Comprensión de los operadores aritméticos fundamentales para estructurar correctamente las reglas de victoria y derrota.

[Documentación](#)

Semana 4: Algoritmos y Diagramas de Flujo

Dominar el diseño lógico es el primer paso fundamental para estructurar las decisiones en nuestro Juego de Piedra, Papel o Tijera.

Diseño Lógico Paso a Paso



Definimos la estructura secuencial del juego, desde la entrada del usuario hasta la validación de resultados, garantizando un flujo coherente y funcional.

Visualización de Decisiones



La creación de diagramas de flujo permite mapear visualmente cada ramificación lógica, facilitando la comprensión de las reglas complejas del juego.

Planificación Estratégica



Antes de escribir una sola línea de código, es vital planificar la lógica para evitar errores estructurales y optimizar el tiempo de desarrollo posterior.

Arquitectura del Juego



Establecer los cimientos mediante algoritmos asegura que el código resultante en Python sea limpio, eficiente y fácil de mantener a largo plazo.

La Importancia del Diseño Previo en Programación

El éxito de nuestro proyecto Python depende directamente de la claridad de nuestros diagramas de flujo. Una buena planificación reduce la frustración durante la codificación y permite prever casos límite en el juego de Piedra, Papel o Tijera. Invertir tiempo en la lógica hoy significa menos errores mañana y un producto final mucho más robusto.

Semana 5: Estructuras de Decisión Controladas

Dominio de la lógica condicional para el desarrollo del proyecto "Piedra, Papel o Tijera" en Python.

Operadores de Comparación



Uso de `==`, `!=`, `<` y `>` para evaluar las elecciones del usuario y la computadora.

Fundamento Lógico 1

Operadores Lógicos



Aplicación de `'and'` y `'or'` para combinar múltiples condiciones en una sola validación.

Fundamento Lógico 2

Sentencias Condicionales



Estructura base de `'if'` para ejecutar acciones cuando se cumple una condición específica.

Control de Flujo 1

Alternativas Elif/Else



Gestión de múltiples caminos lógicos y casos por defecto para capturar errores.

Control de Flujo 2

Implementación de Reglas



Programación detallada de las reglas del juego mediante sentencias anidadas.

- ✓ Verificación de entradas válidas del jugador.
- ✓ Sincronización con la elección aleatoria de la PC.



Lógica de Resultado



Determinación precisa de quién gana, pierde o empata según la jugada seleccionada.

Meta Final:

[Código Funcional](#)

Semana 6: Bucles e Interactividad

En esta etapa implementamos estructuras de control avanzadas para permitir una experiencia de juego continua y dinámica en Python.

1

● Control de Partidas



Implementación del bucle **while** para permitir múltiples partidas continuas sin reiniciar el programa manualmente.

🔁 Iteración infinita controlada

2

● Flujo del Juego



Uso de las sentencias **break** y **continue** para gestionar estados, validar entradas y finalizar el ciclo del juego.

⚙️ Control lógico estructurado

3

● Interfaz de Usuario



Creación de una interfaz interactiva mediante consola que solicita datos al usuario y muestra resultados inmediatos.

💻 Experiencia de usuario (CLI)

4

● Resultado Semanal



Logramos una estructura robusta que permite al jugador decidir cuándo detenerse, integrando toda la lógica previa.

★ Sistema completamente funcional

Semana 7: Estructuras de Datos y Optimización

Consolidación de un código funcional y robusto integrando todos los conocimientos previos del lenguaje Python.

1



● Mejora del Código

Optimizamos el juego utilizando listas y diccionarios para gestionar las opciones y reglas de forma eficiente y escalable.

</>Estructuras eficientes

2



● Registro de Partidas

Implementamos un sistema de historial detallado para almacenar las puntuaciones y resultados de cada partida disputada.

📁Gestión de historial